

```

% Defining paths and leafs (outer nodes) for inference
path(X,Y) :- edge(X,Y).
path(X,Y) :- edge(X,Z),
               Y \= Z,
               path(Z,Y).

% Description of the communicator's ostensive act (utterance)
say(P,S) :- person(P), sentence(S).

% Proposition which is the result of the development of the logical
% form of the utterance = explicature;
% masking a lot of details
% The situation that P does not want to drink X is asserted to be true
% in the world
% if P utters the sentence "I don't drink energizing drinks..."
% and X can be said to be of the category "energizing drink".
wantsNotDrink(P,X) :- say(P,sentence(idontDrinkED)), energizingDrink(X)
.
wantsNotDrink(P,X) :- say(P,sentence(idontDrinkPeD)),
    performanceEnhancingDrink(X).
wantsNotDrink(P,X) :- say(P,sentence(idontDrinkMgD)),
    muscleGrowingDrink(X).
wantsNotDrink(P,X) :- say(P,sentence(idontDrinkCoffee)), X=coffee.

% Some sentences Mary could have said. Need some prior probability less
% than 1,
% otherwise we get a message about inconsistent evidence.
0.01::sentence(idontDrinkED).
0.01::sentence(idontDrinkPeD).
0.01::sentence(idontDrinkMgD).
0.01::sentence(idontDrinkCoffee).

% Sentences Mary may have said.
% Since we want to use a description of what Mary said as evidence,
% we need to assign a certain prior probability to these descriptions,
% otherwise we get an error message about inconsistent evidence.
0.01::say(mary,sentence(idontDrinkED)).
0.01::say(mary,sentence(idontDrinkPeD)).
0.01::say(mary,sentence(idontDrinkMgD)).
0.01::say(mary,sentence(idontDrinkCoffee)).

% Mary provides Peter with partial evidence for her informative
% intention
% by uttering the sentence modeled above. Choose one, or none.
evidence(say(mary,sentence(idontDrinkED))).
% evidence(say(mary,sentence(idontDrinkPeD))).
% evidence(say(mary,sentence(idontDrinkMgD))).
% evidence(say(mary,sentence(idontDrinkCoffee))).

% Inference means querying about the beverages
query(wantsNotDrink(mary,coffee)).
query(wantsNotDrink(mary,tea)).
query(wantsNotDrink(mary,peppermintTea)).
query(wantsNotDrink(mary,fruitTea)).
query(wantsNotDrink(mary,coke)).
query(wantsNotDrink(mary,redBull)).
query(wantsNotDrink(mary,water)).
query(wantsNotDrink(mary,proteinShake)).

```

C Code listing: Int 3

As in Int 1 until line 31. The following probabilistic edges are added at this point:

```
% Hot vs cold drink classification; irrelevant to expectations
0.8::edge(coffee, hotDrink).
0.1::edge(water, hotDrink).
0.8::edge(tea, hotDrink).
0.8::edge(peppermintTea, hotDrink).
0.8::edge(fruitTea, hotDrink).
0.1::edge(coke, hotDrink).
0.1::edge(redBull, hotDrink).
0.2::edge(proteinShake, hotDrink).

0.5::edge(hotDrink, partyDrink).
0.6::edge(hotDrink, tirednessBlockingDrink).
0.4::edge(hotDrink, performanceEnhancingDrink).
0.4::edge(hotDrink, muscleGrowingDrink).
```

D Code listing: Int 4 and Int 5

Int 4 and Int 5 differ from one another only in the extant relevance expectations: Int 5 adds the query `query(healthconscious(mary))`. Other than that Int 4 and Int 5 are identical in their knowledge graphs. The implementation equals that of Int 1 up to line 31. At this point, the following second knowledge graph is added:

```
% Additional encyclopaedic web for potential additional inference
% Whether Mary is healthconscious may be relevant to Peter,
% it could also affect his next move.
0.5::edge(person(mary), healthconscious).
0.2::edge(notDrinkCoffeeBefore6PM, healthconscious).
0.7::edge(notDrinkCoffeeAfter6PM, healthconscious).

edge(person(mary), notDrinkCoffeeAfter6PM) :- wantsNotDrink(mary,
    coffee).
```